

72. Une fonction à toi : bande()

Avant de parler d'#include, on écrit une fonction à soi : bande() pose la même tuile plusieurs fois, de gauche à droite.

1. Ce qu'il faut comprendre

Ce chapitre répond à une question : « comment ajouter moi-même un #include ? » Il y a deux sortes d'#include, et on va les faire toutes les deux, avec **une seule fonction**, du début à la fin : bande().

On part de ce qu'on sait déjà faire (chapitre « Les fonctions : nommer un geste ») : écrire une fonction dans son programme. bande(colonne, ligne, tuile, longueur) pose la tuile longueur fois, une case plus à droite à chaque fois.

Lis la fonction ligne par ligne : void dit qu'elle ne rend rien ; entre les parenthèses, ses quatre paramètres, quatre cases de mémoire remplies par l'appel. La boucle for compte i de 0 jusqu'à longueur - 1, et pose la tuile en colonne + i.

Déroulé de bande(2, 5, ALPHABET[0], 10) : i = 0 → case (2, 5) ; i = 1 → case (3, 5) ; ... ; i = 9 → case (11, 5). Puis i = 10 : 10 < 10 est faux, la boucle s'arrête. Dix A, des colonnes 2 à 11.

Pour l'instant, aucun #include nouveau : la fonction est dans le programme, elle n'a rien à demander à la console. Elle se sert seulement de poser(), déjà incluse.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait 911 octets de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
bande()         38 o ← main() - une fonction fournie par le langage
main()          26 o ← démarrage - ta fonction
VBlank          8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AdresseCase    10 o ← poser() - trouve l'adresse d'une case de l'écran
CopierTuiles   18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser() × 1      17 o sur place, dans bande() + AdresseCase (10 o)
image() × 1       3 o sur place, dans main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
bande() × 1      19 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// Une fonction à toi : bande().
// Elle est écrite ICI, dans le programme : pas besoin d'#include pour elle.

#include <poser>      // pose une tuile sur une case du fond (bande() s'en sert)
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A

// bande(colonne, ligne, tuile, longueur) : la même tuile, « longueur » fois,
// de gauche à droite. Exemple : bande(2, 5, ALPHABET[0], 10) → dix A,
// des colonnes 2 à 11, sur la ligne 5.
//
// void          elle ne rend rien : elle agit, c'est tout
// uint8_t ...    ses quatre paramètres : des nombres de 0 à 255
```

2. UNE FONCTION — BANDE()

```
void bande(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t longueur) {
    // for (départ ; condition ; après chaque tour)
    //   i = 0 au départ ; on continue tant que i < longueur ; i++ ajoute 1.
    for (uint8_t i = 0; i < longueur; i++) {
        poser(colonne + i, ligne, tuile); // une case plus à droite à chaque tour
    }
}
```

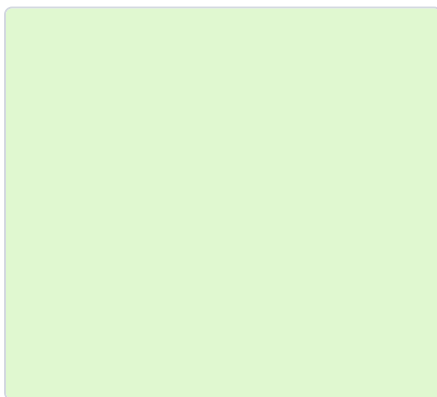
3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    bande(2, 5, ALPHABET[0], 10); // l'appel : dix A sur la ligne 5

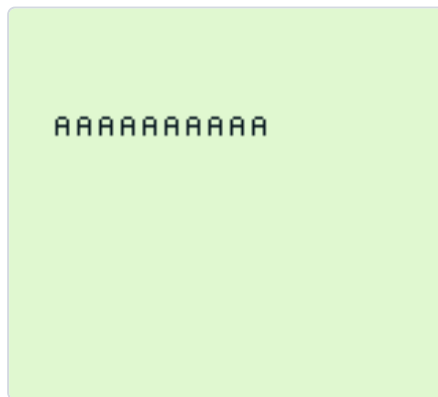
    while (true) { // la boucle du jeu
        image();    // attend l'image suivante (native : pas d'#include)
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Dix A côte à côte sur la ligne 5, des colonnes 2 à 11.

5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ dix A sur la ligne 5, de la colonne 2 à la 11
- ✓ rien avant la colonne 2
- ✓ et rien après la colonne 11

← [71. Une carte plus grande que l'écran : la caméra](#)

[sommaire](#)

[73. La même fonction, trois fois](#) →